Lesson 3: Loops

Loops are used to repeat a block of code. You should understand the concept of C++'s true and false, because it will be necessary when working with loops (the conditions are the same as with if statements). There are three types of loops: for, while, and do..while. Each of them has their specific uses. They are all outlined below.

FOR - for loops are the most useful type. The layout is

```
for ( variable initialization; condition; variable update ) {
  Code to execute while the condition is true
}
```

The variable initialization allows you to either declare a variable and give it a value or give a value to an already existing variable. Second, the condition tells the program that while the conditional expression is true the loop should continue to repeat itself. The variable update section is the easiest way for a for loop to handle changing of the variable. It is possible to do things like x++, x = x + 10, or even x = random (5), and if you really wanted to, you could call other functions that do nothing to the variable but still have a useful effect on the code. Notice that a semicolon separates each of these sections, that is important. Also note that every single one of the sections may be empty, though the semicolons still have to be there. If the condition is empty, it is evaluated as true and the loop will repeat until something else stops it.

Example:

```
#include <iostream>

using namespace std; // So the program can see cout and endl

int main()
{
  // The loop goes while x < 10, and x increases by one every loop
  for ( int x = 0; x < 10; x++ ) {
    // Keep in mind that the loop condition checks
    //  the conditional statement before it loops again.
    //  consequently, when x equals 10 the loop breaks.
    // x is updated before the condition is checked.
    cout<< x <<endl;
  }
  cin.get();
}
```

This program is a very simple example of a for loop. x is set to zero, while x is less than 10 it calls cout<< x <<endl; and it adds 1 to x until the condition is met. Keep in mind also that the variable is incremented after the code in the loop is run for the first time.

WHILE - WHILE loops are very simple. The basic structure is

while (condition) { Code to execute while the condition is true }

The true represents a boolean expression which could be x == 1 or while (x != 7) (x does not equal 7).

It can be any combination of boolean statements that are legal. Even, (while x = =5 || v == 7) which says execute the code while x equals five or while v equals 7. Notice that a while loop is the same as a for loop without the initialization and update sections. However, an empty condition is not legal for a while loop as it is with a for loop.

Example:

```
#include <iostream>

using namespace std; // So we can see cout and endl

int main()
{
  int x = 0;  // Don't forget to declare variables

  while ( x < 10 ) { // While x is less than 10
    cout<< x <<endl;
    x++;             // Update x so the condition can be met eventually
  }
  cin.get();
}
```

This was another simple example, but it is longer than the above FOR loop. The easiest way to think of the loop is that when it reaches the brace at the end it jumps back up to the beginning of the loop, which checks the condition again and decides whether to repeat the block another time, or stop and move to the next statement after the block.

DO..WHILE - DO..WHILE loops are useful for things that want to loop at least once. The structure is

```
do {
} while ( condition );
```

Notice that the condition is tested at the end of the block instead of the beginning, so the block will be executed at least once. If the condition is true, we jump back to the beginning of the block and execute it again. A do..while loop is basically a reversed while loop. A while loop says "Loop while the condition is true, and execute this block of code", a do..while loop says "Execute this block of code, and loop while the condition is true".

Example:

```
#include <iostream>

using namespace std;

int main()
{
  int x;

  x = 0;
  do {
    // "Hello, world!" is printed at least one time
    //  even though the condition is false
```

```
    cout<<"Hello, world!\n";
  } while ( x != 0 );
  cin.get();
}
```

Keep in mind that you must include a trailing semi-colon after the while in the above example. A common error is to forget that a do..while loop must be terminated with a semicolon (the other loops should not be terminated with a semicolon, adding to the confusion). Notice that this loop will execute once, because it automatically executes before checking the condition.